Relatório de Projeto – Metro Mondego

Unidade Curricular: Bases de Dados (2025/2026)**Grupo:** Akcel Soares da Graça, Martim ..., Tiago ...**Data:** 23 de abril de 2026**Versão:** 1.0

1. Introdução

O projeto Metro Mondego consiste no desenvolvimento de um sistema de bilhética e mobilidade urbana, composto por uma base de dados relacional PostgreSQL e uma API REST em Python (Flask). O presente relatório descreve o trabalho realizado até à data, incluindo a modelação conceptual, a implementação do modelo físico, a criação da API com autenticação JWT e os primeiros endpoints funcionais. Todos os componentes foram desenvolvidos em conformidade com os requisitos do enunciado e as boas práticas de desenho de bases de dados.

2. Modelação Conceptual (Diagrama E-R)

O modelo EntidadeRelacionamento foi elaborado na ferramenta **ONDA** (versão 4) e representa todas as entidades, atributos, relacionamentos, cardinalidades e restrições de integridade necessárias para o sistema Metro Mondego.

2.1. Entidades e Atributos

2.1.1. Entidade Pessoa (forte)

Representa qualquer utilizador do sistema.Atributos:

- id (BIGINT, chave primária, gerado automaticamente)
- nome (VARCHAR(512))
- email (VARCHAR(512), único)
- username (VARCHAR(512), único)
- password_hash (VARCHAR(512))

2.1.2. Especialização Cliente (subclasse de Pessoa)

Herda a chave primária id de Pessoa.Atributos específicos:

- wallet (FLOAT(8)) – saldo da carteira
- nif (VARCHAR(512))
- telefone (VARCHAR(512))

Particularidade: especialização total e disjunta (complete). Todo o cliente é uma pessoa; um utilizador não pode ser simultaneamente cliente e administrador.

2.1.3. Especialização Administrador (subclasse de Pessoa)

Herda a chave primária id de Pessoa.Atributo específico:

- is_super (BOOLEAN) – indica se é Super Administrador.

2.1.4. Entidade Linha (forte)

Representa uma linha de metro (fixa, apenas parâmetros operacionais podem ser alterados).Atributos:

- id (BIGINT, PK)

- nome (VARCHAR(512))
- hora_inicio (VARCHAR(512)) – início do serviço no formato 'HH:MM:SS'
- hora_fim (VARCHAR(512)) – fim do serviço
- frequencia (INTEGER) – intervalo entre viagens (minutos)
- capacidade_default (INTEGER) – lotação padrão dos veículos

2.1.5. Entidade Paragem (forte)

Estação ou apeadeiro do metro.Atributos:

- id (BIGINT, PK)
- nome (VARCHAR(512))

2.1.6. Entidade Trajeto (fraca)

Associa uma linha a uma paragem, com atributos que definem a ordem e características do percurso. Depende da entidade forte Linha e referencia Paragem.Atributos:

- sequencia (INTEGER) – ordem da paragem na linha
- tempo_previsto_desde_origem (INTEGER) – minutos desde a origem
- distancia_acumulada (FLOAT(8))
- plataforma_sentido (VARCHAR(512)) – identifica o sentido (ex.: 'A' para ida, 'B' para volta)
- linha_id (BIGINT) – FK para Linha
- paragem_id (BIGINT NOT NULL) – FK para Paragem

Chave primária composta: (sequencia, linha_id, plataforma_sentido). Esta escolha garante que uma mesma linha pode ter múltiplos trajetos (um por sentido e por ordem), incluindo duas "sequências 1" distintas para os sentidos opostos.

2.1.7. Entidade Viagem (forte)

Representa uma partida concreta de uma linha numa data/hora específica.Atributos:

- id (BIGINT, PK)
- data_hora_partida (TIMESTAMP)
- direcao (VARCHAR(512)) – 'ida' ou 'volta'
- capacidade_disponivel (BIGINT) – lugares vagos
- atraso_estimado (BIGINT) – minutos de atraso
- linha_id (BIGINT NOT NULL) – FK para Linha

2.1.8. Entidade Tipo_bilhete (forte)

Catálogo dos tipos de bilhete suportados.Atributos:

- id_tipo (BIGINT, PK)
- nome (VARCHAR(512)) – ex.: 'single_trip', 'daily', 'monthly_pass'

2.1.9. Entidade Historico_Preco (fraca)

Regista o histórico de preços de cada tipo de bilhete, permitindo consultar o preço em vigor em qualquer data. Depende de Tipo_bilhete.Atributos:

- preco (FLOAT(8))
- data_efetiva (DATE) – data a partir da qual o preço é válido
- tipo_bilhete_id_tipo (BIGINT) – FK para Tipo_bilhete

Chave primária composta: (data_efetiva, tipo_bilhete_id_tipo).

2.1.10. Entidade Bilhete (forte)

Representa um bilhete ou passe adquirido por um cliente.Atributos:

- id (BIGINT, PK)
- data_compra (DATE)
- preco_compra (FLOAT(8))
- data_inicio_validade (DATE)
- data_fim_validade (DATE)
- data_viagem (DATE) – para bilhetes single trip com data fixa
- data_expiracao (DATE) – data limite para passes
- estado (VARCHAR(512)) – 'ativo', 'usado', 'expirado', etc.
- metodo_pagamento (VARCHAR(512))
- desconto_aplicado (INTEGER) – desconto em percentagem ou valor
- tipo_bilhete_id_tipo (BIGINT NOT NULL) – FK para Tipo_bilhete
- cliente_pessoa_id (BIGINT NOT NULL) – FK para Cliente

2.1.11. Entidade Validacao (fraca)

Regista a utilização de um bilhete numa viagem, opcionalmente associada a uma paragem. Depende de Bilhete e Viagem.Atributos:

- data_hora (TIMESTAMP)
- bilhete_id (BIGINT) – FK, parte da PK
- viagem_id (BIGINT) – FK, parte da PK
- paragem_id (BIGINT NOT NULL) – FK para Paragem

Chave primária composta: (bilhete_id, viagem_id).

2.1.12. Entidade Promocao (forte)

Regras de desconto aplicáveis a uma linha e a um tipo de bilhete.Atributos:

- ID_promocao (INTEGER, PK)
- nome (VARCHAR(512))
- desconto (INTEGER)
- data_inicio (DATE)
- data_fim (DATE)
- tipo_bilhete_id_tipo (BIGINT NOT NULL) – FK
- linha_id (BIGINT NOT NULL) – FK

2.1.13. Entidade Aviso (forte)

Mensagens enviadas por um administrador.Atributos:

- id (BIGINT, PK)
- titulo, mensagem (VARCHAR(512))
- data_emissao (DATE)
- administrador_pessoa_id (BIGINT NOT NULL) – FK

2.1.14. Entidade Aviso_cliente (fraca)

Associativa entre Cliente e Aviso, indicando se o cliente leu o aviso.Atributos:

- data_entrega (DATE)
- data_leitura (DATE)
- lido (BOOLEAN)
- aviso_id (BIGINT) – FK

- cliente_pessoa_id (BIGINT) – FK

Chave primária composta: (aviso_id, cliente_pessoa_id).

2.1.15. Entidade Carregamento (forte)

Recargas da carteira do cliente.Atributos:

- ID_carregamento (INTEGER, PK)
- valor (FLOAT(8))
- metodo_pagamento (VARCHAR(512))
- data_hora (TIMESTAMP)
- cliente_pessoa_id (BIGINT NOT NULL) – FK

2.1.16. Entidade Interrupcao_Linha (forte)

Fecho temporário de uma linha, criado por um administrador.Atributos:

- ID_interrupcao (INTEGER, PK)
- data_inicio, data_fim (DATE)
- motivo (VARCHAR(512))
- estado (BOOLEAN)
- administrador_pessoa_id (BIGINT NOT NULL) – FK
- linha_id (BIGINT NOT NULL) – FK

2.2. Relacionamentos e Cardinalidades

Os principais relacionamentos definidos no diagrama E-R são:

- Pessoa (0:1) ⊲— (0:1) Cliente (especialização completa)
- Pessoa (0:1) ⊲— (0:1) Administrador (especialização completa)
- Linha (1:1) —tem— (1:n) Trajeto (identificador, entidade fraca)
- Paragem (1:1) —integra— (1:n) Trajeto
- Linha (1:1) —tem— (0:n) Viagem
- Cliente (1:1) —compra— (0:n) Bilhete
- Tipo_bilhete (1:1) —corresponde— (0:n) Bilhete
- Bilhete (1:1) —recebe— (1:n) Validacao (entidade fraca)
- Viagem (1:1) —inclui— (1:n) Validacao (entidade fraca)
- Paragem (1:1) —regista— (0:n) Validacao
- Linha (1:1) —aplica— (0:n) Promocao
- Promocao (0:n) —aplica— (1:1) Tipo_bilhete
- Administrador (1:1) —emite— (0:n) Aviso
- Cliente (1:1) — (0:n) Aviso_cliente (entidade fraca)
- Aviso (1:1) — (0:n) Aviso_cliente (entidade fraca)
- Cliente (1:1) —efetua— (0:n) Carregamento
- Linha (1:1) —numa— (0:n) Interrupcao_Linha
- Administrador (1:1) —cria— (0:n) Interrupcao_Linha

O diagrama E-R completo encontrase na Figura 1 (ficheiro diagrama_conceptual_v3_foto.png) e no ficheiro JSON do ONDA (diagrama_conceptual_final_v3.json).

3. Modelo Relacional (Físico)

A partir do diagrama E-R, foi gerado o modelo físico com as tabelas especificadas no script createtable.sql. As principais características são:

- **Chaves primárias** simples ou compostas, conforme a natureza da entidade (forte ou fraca).
- **Chaves estrangeiras** com as restrições de integridade referencial adequadas.
- **Restrições de unicidade** em pessoa.email e pessoa.username.
- Tipos de dados coerentes com os atributos: BIGINT para IDs, FLOAT(8) para valores monetários, BOOLEAN para flags, VARCHAR(512) para textos, DATE e TIMESTAMP para datas/horas.

O script de criação executa com sucesso no PostgreSQL, incluindo a adição das constraints de FK e UNIQUE após a criação das tabelas.

3.1. Povoamento da Base de Dados

O ficheiro insertdata.sql contém dados de teste coerentes que respeitam a ordem de dependências (entidades fortes primeiro, depois as fracas). Os dados incluem:

- 1 Super Admin, 1 Admin, 5 Clientes
- 3 Linhas (Portagem–Hospital, Portagem–Estacao B, Portagem–Miranda do Corvo–Lousa)
- 6 Paragens (Portagem, Hospital, Estacao B, Miranda do Corvo, Lousa, Serpins)
- 16 Trajetos (duas direções por linha)
- 5 Tipos de bilhete (single_trip, daily, monthly_pass, monthly_student, monthly_senior)
- 5 Históricos de preço (incluindo um aumento futuro)
- 11 Viagens distribuídas pelas 3 linhas
- 2 Promoções, 2 Avisos, 3 Aviso_cliente, 4 Carregamentos, 5 Bilhetes, 6 Validações, 2 Interrupções

Estes dados permitem testar de imediato todos os endpoints previstos no projeto.

4. API REST

A API foi implementada em Python 3, utilizando a biblioteca Flask e o driver psycopg2 para acesso ao PostgreSQL. O ficheiro principal é data-api.py.

4.1. Arquitetura e Configuração

- **Logger**: Configurado ao nível do módulo para registar todas as operações, com saída para ficheiro e consola.
- **Chave secreta JWT**: Definida em app.config['SECRET_KEY'] (para desenvolvimento está em texto simples; em produção seria protegida).
- **Conexão à base de dados**: A função db_connection() estabelece uma ligação sincrona ao PostgreSQL com os parâmetros definidos (host=127.0.0.1, database=metro).

4.2. Mecanismo de Autenticação / Autorização

4.2.1. Geração de tokens JWT (generate_token)

- Recebe user_id, username, is_admin e is_super.
- Cria um payload com esses dados e um campo exp (expira em 2 horas).
- Codifica o payload usando o algoritmo HS256 e a chave secreta.
- Converte o token para string (compatibilidade com versões antigas do PyJWT que podem retornar bytes).

4.2.2. Decorador token_required

- Função que protege os endpoints que exigem autenticação.
- Extrai o token do cabeçalho Authorization: Bearer <token>.
- Se o token estiver ausente, retorna erro 400.

- Se o token estiver expirado ou for inválido, retorna erro 400 com a mensagem adequada.
- Em caso de sucesso, descodifica o token e passa o dicionário current_user ao endpoint.

4.2.3. Endpoint de Login (PUT /dbproj/user)

- Recebe username e password no corpo da requisição (JSON).
- Consulta a tabela pessoa com LEFT JOIN administrador para obter as flags is_admin e is_super.
- Compara a password recebida com a armazenada (neste momento, comparação direta porque os dados de teste estão em texto simples).
- Se válido, gera e devolve o token JWT em results.
- Em caso de credenciais inválidas, retorna erro 400.

4.2.4. Endpoint de Criação de Administrador (PUT /dbproj/register/admin)

- Protegido com @token_required e restrito a is_super == True.
- Valida que os campos name, email e password estão presentes.
- Define username = email (decisão de design – o username é igual ao email).
- Verifica se o email/username já existe na base de dados.
- Calcula o próximo id como MAX(id)+1 (simulação de autoincremento).
- Insere um novo registo em pessoa e outro em administrador (is_super = FALSE).
- Em caso de sucesso, devolve 200 com {"results": {"user_id": <id>}}.
- Em caso de violação de integridade, retorna erro 400.

4.3. Endpoints implementados e testados

Método	URL	Descrição	Autenticação	Autorização
PUT	/dbproj/user	Login e obtenção de token JWT	Não	Não
PUT	/dbproj/register/admin	Criar novo administrador	Token obrigatório	Super Admin
GET	/	Landing page (informação)	Não	Não

5. Testes e Validação

5.1. Ambiente de Teste

- **Servidor**: Flask em modo debug (autorecarregamento) no endereço http://127.0.0.1:8080.
- **Base de dados**: PostgreSQL local, base metro povoada com o script insertdata.sql.
- **Ferramenta de teste**: Postman.

5.2. Resultados Obtidos (Logs)

O ficheiro log_file.log regista todas as interações durante a fase de desenvolvimento. Destacamse os seguintes eventos:

1. **Login com sucesso**PUT /dbproj/user com "username":"superadmin","password":"hash_super"Resposta: 200

log

DEBUG:logger:Login bem-sucedido para superadmin, admin=True, super=True

1. **Tentativa de criação de admin sem token**Várias requisições PUT /dbproj/register/admin sem cabeçalho Authorization resultaram em 400 "Token em falta".
2. **Autenticação do cliente brunosilva**Login com sucesso, admin=False, super=None. (Nota: o campo is_super aparece como nullno token para clientes normais, o que é esperado do LEFT JOIN).
3. **Criação de administrador com sucesso**Após correções no código, o log mostra:

log

DEBUG:logger:Authorization header recebido: Bearer eyJ...

DEBUG:logger:Token válido para user=superadmin

DEBUG:logger:Administrador criado: id=8, nome=Novo Admin, email=admin@exemplo.pt

A resposta foi 200 e o novo administrador foi inserido com id=8.

1. **Teste de permissão**Um token de cliente (anacosta) foi usado para tentar criar um administrador. O sistema recusou com 400:

log

WARNING:logger:Acesso negado para anacosta

1. **Token inválido**Foram detetadas tentativas com token adulterado (assinatura inválida), resultando em 400"Token inválido".

Estes resultados comprovam que a autenticação JWT e a autorização baseada em permissões (is_super) estão a funcionar conforme o especificado.

6. Decisões de Design e Justificações

1. **Chave primária composta em trajeto**A inclusão de plataforma_sentido na PK permite que a mesma linha tenha trajetos de ida e volta com a mesma sequencia, mas em sentidos opostos, sem violar a unicidade.
2. **Username igual ao email**Na criação de administradores (e futuramente de clientes), o campo username é preenchido automaticamente com o email. Isto simplifica o registo e evita ambiguidades, já que o login pode ser feito indiferentemente por qualquer um dos dois; contudo, a tabela pessoa mantém ambos os campos separados para flexibilidade.
3. **Geração manual de IDs**Como as tabelas não utilizam SERIAL, o próximo id é obtido com SELECT COALESCE(MAX(id), 0) + 1. Esta abordagem é suficiente para o ambiente de desenvolvimento, mas sera substituída por sequências na versão final para garantir atomicidade e desempenho.
4. **Comparação direta de passwords**Os dados de teste armazenam as passwords em texto simples (hash_super, etc.). Para o checkpoint atual, a verificação é feita por igualdade simples. Numa versão de produção, seria implementado hashing com bcrypt e o login usaria check_password_hash.
5. **Log detalhado**A colocação de mensagens de debug no decorador token_required e nos endpoints facilitou a depuração dos erros de "Token em falta" encontrados nos primeiros

testes, permitindo confirmar que o cabeçalho estava a ser recebido mas o token não era extraído corretamente do Authorization: Bearer <token>.

7. Próximos Passos

Com o checkpoint 1 concluído (autenticação + endpoint básico de criação de admin), os trabalhos seguintes incluem:

- Implementar os restantes endpoints obrigatórios (customer, line operation, fares, notices, promotions, purchase, validation, reports).
- Adicionar triggers no PostgreSQL para deduzir saldo na compra de bilhetes e decrementar capacidade na validação.
- Tratar concorrência com transações explícitas e tratamento de serialization failure.
- Elaborar o manual de instalação e a coleção Postman completa.

8. Conclusão

Até ao momento, o projeto Metro Mondego conta com uma base de dados normalizada, uma API REST funcional com autenticação JWT, e dois endpoints operacionais que validam os mecanismos de autorização. O modelo E-R está corretamente mapeado para o modelo relacional, e os dados de teste permitem verificar as funcionalidades implementadas. O código é modular e preparado para receber os restantes endpoints. A equipa está confiante de que o sistema cumprirá todos os requisitos do enunciado até à entrega final.

Relatório de Projeto – Metro Mondego

Unidade Curricular: Bases de Dados (2025/2026)**Grupo:** Akcel Soares da Graça, Martim ..., Tiago ...**Data:** 23 de abril de 2026**Versão:** 1.0

1. Introdução

O projeto Metro Mondego consiste no desenvolvimento de um sistema de bilhética e mobilidade urbana, composto por uma base de dados relacional PostgreSQL e uma API REST em Python (Flask). O presente relatório descreve o trabalho realizado até à data, incluindo a modelação conceptual, a implementação do modelo físico, a criação da API com autenticação JWT e os primeiros endpoints funcionais. Todos os componentes foram desenvolvidos em conformidade com os requisitos do enunciado e as boas práticas de desenho de bases de dados.

2. Modelação Conceptual (Diagrama E-R)

O modelo EntidadeRelacionamento foi elaborado na ferramenta **ONDA** (versão 4) e representa todas as entidades, atributos, relacionamentos, cardinalidades e restrições de integridade necessárias para o sistema Metro Mondego.

2.1. Entidades e Atributos

2.1.1. Entidade Pessoa (forte)

Representa qualquer utilizador do sistema.Atributos:

- id (BIGINT, chave primária, gerado automaticamente)
- nome (VARCHAR(512))
- email (VARCHAR(512), único)
- username (VARCHAR(512), único)
- password_hash (VARCHAR(512))

2.1.2. Especialização Cliente (subclasse de Pessoa)

Herda a chave primária id de Pessoa.Atributos específicos:

- wallet (FLOAT(8)) – saldo da carteira
- nif (VARCHAR(512))
- telefone (VARCHAR(512))

Particularidade: especialização total e disjunta (complete). Todo o cliente é uma pessoa; um utilizador não pode ser simultaneamente cliente e administrador.

2.1.3. Especialização Administrador (subclasse de Pessoa)

Herda a chave primária id de Pessoa.Atributo específico:

- is_super (BOOLEAN) – indica se é Super Administrador.

2.1.4. Entidade Linha (forte)

Representa uma linha de metro (fixa, apenas parâmetros operacionais podem ser alterados).Atributos:

- id (BIGINT, PK)
- nome (VARCHAR(512))
- hora_inicio (VARCHAR(512)) – início do serviço no formato 'HH:MM:SS'
- hora_fim (VARCHAR(512)) – fim do serviço
- frequencia (INTEGER) – intervalo entre viagens (minutos)
- capacidade_default (INTEGER) – lotação padrão dos veículos

2.1.5. Entidade Paragem (forte)

Estação ou apeadeiro do metro.Atributos:

- id (BIGINT, PK)
- nome (VARCHAR(512))

2.1.6. Entidade Trajeto (fraca)

Associa uma linha a uma paragem, com atributos que definem a ordem e características do percurso. Depende da entidade forte Linha e referencia Paragem.Atributos:

- sequencia (INTEGER) – ordem da paragem na linha
- tempo_previsto_desde_origem (INTEGER) – minutos desde a origem
- distancia_acumulada (FLOAT(8))

- plataforma_sentido (VARCHAR(512)) – identifica o sentido (ex.: 'A' para ida, 'B' para volta)
- linha_id (BIGINT) – FK para Linha
- paragem_id (BIGINT NOT NULL) – FK para Paragem

Chave primária composta: (sequencia, linha_id, plataforma_sentido). Esta escolha garante que uma mesma linha pode ter múltiplos trajetos (um por sentido e por ordem), incluindo duas "sequências 1" distintas para os sentidos opostos.

2.1.7. Entidade Viagem (forte)

Representa uma partida concreta de uma linha numa data/hora específica.Atributos:

- id (BIGINT, PK)
- data_hora_partida (TIMESTAMP)
- direcao (VARCHAR(512)) – 'ida' ou 'volta'
- capacidade_disponivel (BIGINT) – lugares vagos
- atraso_estimado (BIGINT) – minutos de atraso
- linha_id (BIGINT NOT NULL) – FK para Linha

2.1.8. Entidade Tipo_bilhete (forte)

Catálogo dos tipos de bilhete suportados.Atributos:

- id_tipo (BIGINT, PK)
- nome (VARCHAR(512)) – ex.: 'single_trip', 'daily', 'monthly_pass'

2.1.9. Entidade Historico_Preco (fraca)

Regista o histórico de preços de cada tipo de bilhete, permitindo consultar o preço em vigor em qualquer data. Depende de Tipo_bilhete.Atributos:

- preco (FLOAT(8))
- data_efetiva (DATE) – data a partir da qual o preço é válido
- tipo_bilhete_id_tipo (BIGINT) – FK para Tipo_bilhete

Chave primária composta: (data_efetiva, tipo_bilhete_id_tipo).

2.1.10. Entidade Bilhete (forte)

Representa um bilhete ou passe adquirido por um cliente.Atributos:

- id (BIGINT, PK)
- data_compra (DATE)
- preco_compra (FLOAT(8))
- data_inicio_validade (DATE)
- data_fim_validade (DATE)
- data_viagem (DATE) – para bilhetes single trip com data fixa
- data_expiracao (DATE) – data limite para passes
- estado (VARCHAR(512)) – 'ativo', 'usado', 'expirado', etc.
- metodo_pagamento (VARCHAR(512))
- desconto_aplicado (INTEGER) – desconto em percentagem ou valor

- tipo_bilhete_id_tipo (BIGINT NOT NULL) – FK para Tipo_bilhete
- cliente_pessoa_id (BIGINT NOT NULL) – FK para Cliente

2.1.11. Entidade Validacao (fraca)

Regista a utilização de um bilhete numa viagem, opcionalmente associada a uma paragem. Depende de Bilhete e Viagem.Atributos:

- data_hora (TIMESTAMP)
- bilhete_id (BIGINT) – FK, parte da PK
- viagem_id (BIGINT) – FK, parte da PK
- paragem_id (BIGINT NOT NULL) – FK para Paragem

Chave primária composta: (bilhete_id, viagem_id).

2.1.12. Entidade Promocao (forte)

Regras de desconto aplicáveis a uma linha e a um tipo de bilhete.Atributos:

- ID_promocao (INTEGER, PK)
- nome (VARCHAR(512))
- desconto (INTEGER)
- data_inicio (DATE)
- data_fim (DATE)
- tipo_bilhete_id_tipo (BIGINT NOT NULL) – FK
- linha_id (BIGINT NOT NULL) – FK

2.1.13. Entidade Aviso (forte)

Mensagens enviadas por um administrador.Atributos:

- id (BIGINT, PK)
- titulo, mensagem (VARCHAR(512))
- data_emissao (DATE)
- administrador_pessoa_id (BIGINT NOT NULL) – FK

2.1.14. Entidade Aviso_cliente (fraca)

Associativa entre Cliente e Aviso, indicando se o cliente leu o aviso.Atributos:

- data_entrega (DATE)
- data_leitura (DATE)
- lido (BOOLEAN)
- aviso_id (BIGINT) – FK
- cliente_pessoa_id (BIGINT) – FK

Chave primária composta: (aviso_id, cliente_pessoa_id).

2.1.15. Entidade Carregamento (forte)

Recargas da carteira do cliente.Atributos:

- ID_carregamento (INTEGER, PK)
- valor (FLOAT(8))
- metodo_pagamento (VARCHAR(512))
- data_hora (TIMESTAMP)
- cliente_pessoa_id (BIGINT NOT NULL) – FK

2.1.16. Entidade Interrupcao_Linha (forte)

Fecho temporário de uma linha, criado por um administrador.Atributos:

- ID_interrupcao (INTEGER, PK)
- data_inicio, data_fim (DATE)
- motivo (VARCHAR(512))
- estado (BOOLEAN)
- administrador_pessoa_id (BIGINT NOT NULL) – FK
- linha_id (BIGINT NOT NULL) – FK

2.2. Relacionamentos e Cardinalidades

Os principais relacionamentos definidos no diagrama E-R são:

- Pessoa (0:1) ⊲— (0:1) Cliente (especialização completa)
- Pessoa (0:1) ⊲— (0:1) Administrador (especialização completa)
- Linha (1:1) —tem— (1:n) Trajeto (identificador, entidade fraca)
- Paragem (1:1) —integra— (1:n) Trajeto
- Linha (1:1) —tem— (0:n) Viagem
- Cliente (1:1) —compra— (0:n) Bilhete
- Tipo_bilhete (1:1) —corresponde— (0:n) Bilhete
- Bilhete (1:1) —recebe— (1:n) Validacao (entidade fraca)
- Viagem (1:1) —inclui— (1:n) Validacao (entidade fraca)
- Paragem (1:1) —regista— (0:n) Validacao
- Linha (1:1) —aplica— (0:n) Promocao
- Promocao (0:n) —aplica— (1:1) Tipo_bilhete
- Administrador (1:1) —emite— (0:n) Aviso
- Cliente (1:1) — (0:n) Aviso_cliente (entidade fraca)
- Aviso (1:1) — (0:n) Aviso_cliente (entidade fraca)
- Cliente (1:1) —efetua— (0:n) Carregamento
- Linha (1:1) —numa— (0:n) Interrupcao_Linha
- Administrador (1:1) —cria— (0:n) Interrupcao_Linha

O diagrama E-R completo encontrase na Figura 1 (ficheiro diagrama_conceptual_v3_foto.png) e no ficheiro JSON do ONDA (diagrama_conceptual_final_v3.json).

3. Modelo Relacional (Físico)

A partir do diagrama E-R, foi gerado o modelo físico com as tabelas especificadas no script createtable.sql. As principais características são:

- **Chaves primárias** simples ou compostas, conforme a natureza da entidade (forte ou fraca).

- **Chaves estrangeiras** com as restrições de integridade referencial adequadas.
- **Restrições de unicidade** em pessoa.email e pessoa.username.
- Tipos de dados coerentes com os atributos: BIGINT para IDs, FLOAT(8) para valores monetários, BOOLEAN para flags, VARCHAR(512) para textos, DATE e TIMESTAMP para datas/horas.

O script de criação executa com sucesso no PostgreSQL, incluindo a adição das constraints de FK e UNIQUE após a criação das tabelas.

3.1. Povoamento da Base de Dados

O ficheiro insertdata.sql contém dados de teste coerentes que respeitam a ordem de dependências (entidades fortes primeiro, depois as fracas). Os dados incluem:

- 1 Super Admin, 1 Admin, 5 Clientes
- 3 Linhas (Portagem–Hospital, Portagem–Estacao B, Portagem–Miranda do Corvo–Lousa)
- 6 Paragens (Portagem, Hospital, Estacao B, Miranda do Corvo, Lousa, Serpins)
- 16 Trajetos (duas direções por linha)
- 5 Tipos de bilhete (single_trip, daily, monthly_pass, monthly_student, monthly_senior)
- 5 Históricos de preço (incluindo um aumento futuro)
- 11 Viagens distribuídas pelas 3 linhas
- 2 Promoções, 2 Avisos, 3 Aviso_cliente, 4 Carregamentos, 5 Bilhetes, 6 Validações, 2 Interrupções

Estes dados permitem testar de imediato todos os endpoints previstos no projeto.

4. API REST

A API foi implementada em Python 3, utilizando a biblioteca Flask e o driver psycopg2 para acesso ao PostgreSQL. O ficheiro principal é data-api.py.

4.1. Arquitetura e Configuração

- **Logger**: Configurado ao nível do módulo para registar todas as operações, com saída para ficheiro e consola.
- **Chave secreta JWT**: Definida em app.config['SECRET_KEY'] (para desenvolvimento está em texto simples; em produção seria protegida).
- **Conexão à base de dados**: A função db_connection() estabelece uma ligação sincrona ao PostgreSQL com os parâmetros definidos (host=127.0.0.1, database=metro).

4.2. Mecanismo de Autenticação / Autorização

4.2.1. Geração de tokens JWT (generate_token)

- Recebe user_id, username, is_admin e is_super.
- Cria um payload com esses dados e um campo exp (expira em 2 horas).
- Codifica o payload usando o algoritmo HS256 e a chave secreta.
- Converte o token para string (compatibilidade com versões antigas do PyJWT que podem retornar bytes).

4.2.2. Decorador token_required

- Função que protege os endpoints que exigem autenticação.
- Extrai o token do cabeçalho Authorization: Bearer <token>.
- Se o token estiver ausente, retorna erro 400.
- Se o token estiver expirado ou for inválido, retorna erro 400 com a mensagem adequada.
- Em caso de sucesso, descodifica o token e passa o dicionário current_user ao endpoint.

4.2.3. Endpoint de Login (PUT /dbproj/user)

- Recebe username e password no corpo da requisição (JSON).
- Consulta a tabela pessoa com LEFT JOIN administrador para obter as flags is_admin e is_super.
- Compara a password recebida com a armazenada (neste momento, comparação direta porque os dados de teste estão em texto simples).
- Se válido, gera e devolve o token JWT em results.
- Em caso de credenciais inválidas, retorna erro 400.

4.2.4. Endpoint de Criação de Administrador (PUT /dbproj/register/admin)

- Protegido com @token_required e restrito a is_super == True.
- Valida que os campos name, email e password estão presentes.
- Define username = email (decisão de design – o username é igual ao email).
- Verifica se o email/username já existe na base de dados.
- Calcula o próximo id como MAX(id)+1 (simulação de autoincremento).
- Insere um novo registo em pessoa e outro em administrador (is_super = FALSE).
- Em caso de sucesso, devolve 200 com {"results": {"user_id": <id>}}.
- Em caso de violação de integridade, retorna erro 400.

4.3. Endpoints implementados e testados

Método	URL	Descrição	Autenticação	Autorização
PUT	/dbproj/user	Login e obtenção de token JWT	Não	Não
PUT	/dbproj/register/admin	Criar novo administrador	Token obrigatório	Super Admin
GET	/	Landing page (informação)	Não	Não

5. Testes e Validação

5.1. Ambiente de Teste

- **Servidor**: Flask em modo debug (autorecarregamento) no endereço http://127.0.0.1:8080.
- **Base de dados**: PostgreSQL local, base metro povoada com o script insertdata.sql.
- **Ferramenta de teste**: Postman.

5.2. Resultados Obtidos (Logs)

O ficheiro log_file.log regista todas as interações durante a fase de desenvolvimento. Destacamse os seguintes eventos:

1. **Login com sucesso**PUT /dbproj/user com "username":"superadmin","password":"hash_super"Resposta: 200

log

DEBUG:logger:Login bem-sucedido para superadmin, admin=True, super=True

1. **Tentativa de criação de admin sem token**Várias requisições PUT /dbproj/register/admin sem cabeçalho Authorization resultaram em 400 "Token em falta".
2. **Autenticação do cliente brunosilva**Login com sucesso, admin=False, super=None. (Nota: o campo is_super aparece como nullno token para clientes normais, o que é esperado do LEFT JOIN).
3. **Criação de administrador com sucesso**Após correções no código, o log mostra:

log

DEBUG:logger:Authorization header recebido: Bearer eyJ...

DEBUG:logger:Token válido para user=superadmin

DEBUG:logger:Administrador criado: id=8, nome=Novo Admin, email=admin@exemplo.pt

A resposta foi 200 e o novo administrador foi inserido com id=8.

1. **Teste de permissão**Um token de cliente (anacosta) foi usado para tentar criar um administrador. O sistema recusou com 400:

log

WARNING:logger:Acesso negado para anacosta

1. **Token inválido**Foram detetadas tentativas com token adulterado (assinatura inválida), resultando em 400"Token inválido".

Estes resultados comprovam que a autenticação JWT e a autorização baseada em permissões (is_super) estão a funcionar conforme o especificado.

6. Decisões de Design e Justificações

1. **Chave primária composta em trajeto**A inclusão de plataforma_sentido na PK permite que a mesma linha tenha trajetos de ida e volta com a mesma sequencia, mas em sentidos opostos, sem violar a unicidade.

2. **Username igual ao email**Na criação de administradores (e futuramente de clientes), o campo username é preenchido automaticamente com o email. Isto simplifica o registo e evita ambiguidades, já que o login pode ser feito indiferentemente por qualquer um dos dois; contudo, a tabela pessoa mantém ambos os campos separados para flexibilidade.
3. **Geração manual de IDs**Como as tabelas não utilizam SERIAL, o próximo id é obtido com SELECT COALESCE(MAX(id), 0) + 1. Esta abordagem é suficiente para o ambiente de desenvolvimento, mas sera substituída por sequências na versão final para garantir atomicidade e desempenho.
4. **Comparação direta de passwords**Os dados de teste armazenam as passwords em texto simples (hash_super, etc.). Para o checkpoint atual, a verificação é feita por igualdade simples. Numa versão de produção, seria implementado hashing com bcrypt e o login usaria check_password_hash.
5. **Log detalhado**A colocação de mensagens de debug no decorador token_required e nos endpoints facilitou a depuração dos erros de "Token em falta" encontrados nos primeiros testes, permitindo confirmar que o cabeçalho estava a ser recebido mas o token não era extraído corretamente do Authorization: Bearer <token>.

7. Próximos Passos

Com o checkpoint 1 concluído (autenticação + endpoint básico de criação de admin), os trabalhos seguintes incluem:

- Implementar os restantes endpoints obrigatórios (customer, line operation, fares, notices, promotions, purchase, validation, reports).
- Adicionar triggers no PostgreSQL para deduzir saldo na compra de bilhetes e decrementar capacidade na validação.
- Tratar concorrência com transações explícitas e tratamento de serialization failure.
- Elaborar o manual de instalação e a coleção Postman completa.

8. Conclusão

Até ao momento, o projeto Metro Mondego conta com uma base de dados normalizada, uma API REST funcional com autenticação JWT, e dois endpoints operacionais que validam os mecanismos de autorização. O modelo E-R está corretamente mapeado para o modelo relacional, e os dados de teste permitem verificar as funcionalidades implementadas. O código é modular e preparado para receber os restantes endpoints. A equipa está confiante de que o sistema cumprirá todos os requisitos do enunciado até à entrega final.

9. Problemas Encontrados

9.1. Chave primária incorreta na tabela trajeto

- **Sintoma**: Ao inserir os primeiros trajetos, o PostgreSQL rejeitava com duplicate key value violates unique constraint "trajeto_pkey" quando se tentava adicionar uma segunda paragem na mesma linha.
- **Causa**: A PK da tabela trajeto estava definida apenas como (linha_id), o que impedia que uma linha tivesse mais do que uma paragem.

- **Diagnóstico**: Revisão do createtable.sql gerado pelo ONDA; notouse que o atributo sequencia não fazia parte da PK.
- **Solução**: Alterouse a PK para (sequencia, linha_id, plataforma_sentido), garantindo que uma linha possa ter múltiplos trajetos, um para cada sentido, e que a mesma sequência possa existir nos dois sentidos.

9.2. Ordem de inserção de dados e violação de dependências

- **Sintoma**: Erros de foreign key ao executar o insertdata.sql porque se tentava inserir primeiro em trajeto (entidade fraca) antes de povoar linha e paragem.
- **Causa**: Desrespeito pela hierarquia de dependências; entidades fracas precisam que as fortes já existam.
- **Solução**: Reorganização completa do script de inserção, agrupando as instruções por níveis (Nível 1: entidades independentes; Nível 2: dependentes apenas do Nível 1; Nível 3: dependentes de níveis anteriores, etc.). Esta ordenação passou a garantir que todas as FKs são satisfeitas no momento da inserção.

9.3. Token JWT não encontrado ("Token em falta")

- **Sintoma**: O endpoint PUT /dbproj/register/admin devolvia consistentemente 400 com "errors": "Token em falta", embora o token tivesse sido gerado no login.
- **Causa**: O pedido HTTP não incluía o cabeçalho Authorization: Bearer <token>; inicialmente, o token era enviado apenas no corpo da requisição, mas o decorador token_required só o procurava no cabeçalho.
- **Solução**: Correcção dos testes no Postman para adicionar o cabeçalho Authorization com o prefixo Bearer. Adicionalmente, melhorouse o código do decorador para registar o cabeçalho recebido (via logger.debug), o que permitiu verificar rapidamente que o problema era a ausência do cabeçalho e não um erro de validação do token.

9.4. Tokens com is_super = null para clientes

- **Sintoma**: Nos logs, após o login de um cliente normal, o token gerado continha is_super = null. Embora não causasse erros, a comparação current_user.get('is_super')podia falhar se não fosse tratada como None.
- **Causa**: A query SQL usava a.is_super sem COALESCE. Como os clientes não têm registo em administrador, o LEFT JOIN devolvia NULL.
- **Solução**: Aplicação de COALESCE(a.is_super, FALSE) AS is_super na query de login e, em alternativa, uso de current_user.get('is_super') que retorna None quando a chave não existe, mas que é tratado como falso no if not current_user.get(...). Optouse por manter a verificação com get() para ambos os casos, o que tornou o código robusto.

9.5. Geração manual de IDs e risco de concorrência

- **Contexto**: Como a tabela pessoa não tinha SERIAL, o próximo id era obtido com SELECT MAX(id)+1.
- **Risco**: Em ambiente concorrente, duas transações simultâneas poderiam obter o mesmo MAX(id) e tentar inserir o mesmo id, causando duplicate key.
- **Mitigação temporária**: Para o checkpoint atual, o sistema é usado por um único utilizador de cada vez, pelo que o risco é negligenciável. No entanto, registouse a necessidade de migrar para sequências (SERIAL ou BIGSERIAL) antes da entrega final.

9.6. Passwords em texto simples nos dados de teste

- **Risco**: As passwords armazenadas como hash_super, hash_admin1 são vulneráveis a roubo.
- **Decisão consciente**: Para facilitar os testes iniciais e o desenvolvimento rápido, usaramse strings simples. O código de login, contudo, foi preparado para comparar hashes Werkzeug,

bastando ativar a geração segura de passwords no registo. A migração para hashing com bcrypt está planeada para a versão final.

10. Lições Aprendidas

1. **Modelação conceptual rigorosa evita retrabalho**A decisão de incluir plataforma_sentido na PK de Trajeto só surgiu após a deteção do erro de duplicação. Uma análise mais cuidada dos requisitos (dois sentidos por linha) teria evitado a correção posterior. Aprendemos a validar exaustivamente os requisitos de unicidade antes de gerar o modelo físico.
2. **Dependências de dados devem ser explícitas e automatizadas**A ordem de inserção manual é propensa a erros. Descobrimos que, em projetos reais, scripts de povoamento devem ser gerados automaticamente a partir do modelo ou, no mínimo, organizados com comentários que indiquem os níveis de dependência.
3. **Autenticação JWT exige atenção aos cabeçalhos HTTP**O erro "Token em falta" ensinounos que, apesar de o enunciado permitir enviar o token no corpo, a prática mais comum (e a que o nosso decorador implementou) é usar o cabeçalho Authorization. Aprendemos a configurar corretamente o Postman e a incluir mensagens de debug que mostram exatamente o que o servidor recebeu.
4. **Logs são ferramenta indispensável de debug**A introdução de logger.debug no decorador token_required e nos endpoints permitiunos perceber rapidamente se o token estava a ser extraído, se era válido e qual o utilizador associado. Sem esses logs, a depuração teria sido muito mais demorada.
5. **Tratamento de casos NULL em SQL é crítico**O LEFT JOIN com administrador devolvia NULL para clientes, o que causava o campo is_super = null no token. Aprendemos a usar COALESCE para garantir que as expressões booleanas nunca sejam NULL, evitando comportamentos imprevisíveis.
6. **Desenvolvimento incremental com verificação contínua**A abordagem de implementar um endpoint de cada vez e testálo imediatamente no Postman, analisando os logs, foi fundamental para identificar e corrigir problemas rapidamente. Esta metodologia será mantida para os restantes endpoints.
7. **Flexibilidade do código para evolução futura**Mesmo usando passwords em texto simples nos testes, o código de login já está preparado para suportar hashing seguro. Esta separação entre a lógica de negócio e os dados de teste revelouse uma boa prática, pois permite transitar para um ambiente mais seguro sem alterar a API.